



Version: 11.0

Trainer:

Course Code: TGS-2021008635

Website: www.tertiarycourses.com.sg

Email: enquiry@tertiaryinfotech.com

Tel: +65 6100 0613

Digital Attendance

- It is mandatory for you to take both AM, PM and Assessment digital attendance for WSQ funded courses.
- The trainer or administrator will show you the digital attendance QR code generated from SSG portal.
- Please scan the QR code from your mobile phone camera and submit your attendance.

About the Trainer

Shahul H. Maricar has been a content developer and webmaster, building educational websites and applications with HTML, CSS and JavaScript. He then served as an IT analyst, writing programs for automating custom workflows as well as data extraction and analysis in the healthcare field.

He is currently a freelance educator and is actively involved with development projects in game programming, computer -aided design and computer graphics.



Let's Know Each Other...



Ground Rules

- Set your mobile phone to silent mode
- Participate actively in the class. No question is stupid.
- Mutual respect. Agree to disagree.
- One conversation at one time.
- Be punctual. Back from breaks on time.
- Exit the class silently if you need to step out for phone call, toilet break etc.
- 75% attendance is required

Lesson Plan

Day 1

- **Digital Attendance (AM)**
- Trainer and Learners Introduction
- Learning Outcomes
- Course Outline
- Topic 1 Get Started on Vue.js
- Topic 2 Using Vue Directives
- Lunch Break
- **Digital Attendance (PM)**
- Topic 3 Controls and Elements
- End of Day 1

Lesson Plan

Day 2

- **Digital Attendance (AM)**
- Topic 4 Components
- Lunch Break
- **Digital Attendance (PM)**
- Topic 5 Create User Defined Forms
- **Course Feedback and TRAQOM Survey**
- **Digital Attendance (Assessment)**
- Final Assessment
- End of Class

Skills Framework

TSC Title: Software Design

TSC Code: ICT-DES-3005-1.1 TSC

Skills Framework TSC

TSC Abilities:

A1: Design a simple software component or interface according to functional specifications and business requirements

A2: Utilise appropriate software design methods and tools, in line with the organisation's software design practice and principles

A3: Identify relevant controls, elements and features to be included in the software to meet its design objectives

A4: Assess functionality and interoperability of different elements or components in the software design

A5: Produce detailed design documentation mapped to user specifications

TSC Knowledge:

K1: Design requirements for simple, basic software components

K2: Basic software design tools and techniques

K3: Types of controls, elements and features in software

K4: Indicators of software functionality and interoperability

K5: Documentation of design details

Learning Outcomes

- LO1: Design a simple Vue App interface
- LO2: Utilize Vue directives
- LO3: Create Vue elements and controls to meet design objectives
- LO4: Create Vue components in the software design
- LO5: Produce Vue forms and documentation

Course Outline

Topic 1 Get Started on Vue.js (A1, K1)

- Overview of Vue.js
- Install Vue.js
- Create a Vue App

Topic 2 Using Vue Directives (A2, K2)

- Attribute Binding
- Conditional Rendering
- List Rendering

Topic 3 Controls and Elements (A3, K3)

- Event Handling
- Class and Styling Bindings
- Computed Properties

Course Outline

Topic 4 Components (A4, K4)

- Components
- Props
- Slots
- Communicating Events

Topic 5 Create User Defined Forms (A4, K4)

- Forms
- Form Input Bindings

Final Assessment

- **Written Assessment (Short Answer Questions)**
- **Practical Performance**
- Assessment format: Open Book
- Open book assessment **ONLY** includes Slides, Learner Guide or any approved materials.
- Appeal process

Briefing for Assessment

- Place phones & other materials under the table or on the floor
- No photos or recording of assessment scripts
- No discussion during assessment
- Use black/blue pen for assessment [hard copies]
- No usage of liquid paper or correction tape
- Assessment scripts will be collected when the time is up

Criteria for Funding

- Minimum attendance rate of 75% based on SSG Digital Attendance record.
- Complete the assessment and be assessed as 'Competent'

LMS/TMS

<https://lms-tms.tertiaryinfotech.com>

This material belongs to Tertiary Infotech Academy Pte



Tertiary Infotech Academy

LMS cum TMS for WSQ Courses

Email



Send OTP

[Login with password instead](#)

[Privacy Policy](#) [Acceptable Use Policy](#) [Feedback](#)

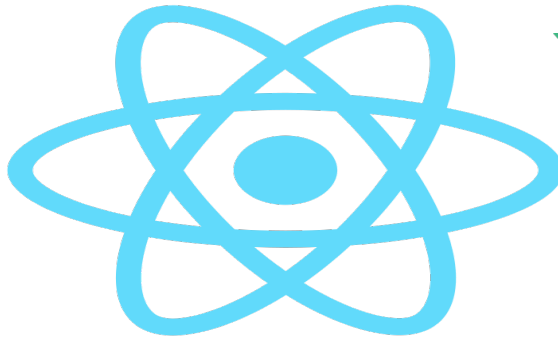
Browse our WSQ courses at
<https://www.tertiarycourses.com.sg/>

Topic 1

Overview of Vue JS

Javascript Frameworks for Web

- Javascript frameworks for Web shift the some of heavy lifting tasks, traditionally done by server scripting language such as PHP, from server to browser
- There are 3 main Javascript frameworks for web



What is Vue.s

- Vue.js is a progressive JavaScript framework used to develop interactive web interfaces.
- Vue JS is preferred due to:
 - Easy to Use. Vue JS does not use Typescript and ES6
 - Fast and Lightweight
 - Reusable Components
 - Scalable

History of Vue

- Vue was created by Evan You after working for Google using AngularJS in a number of projects
- Vue was originally released in February 2014.

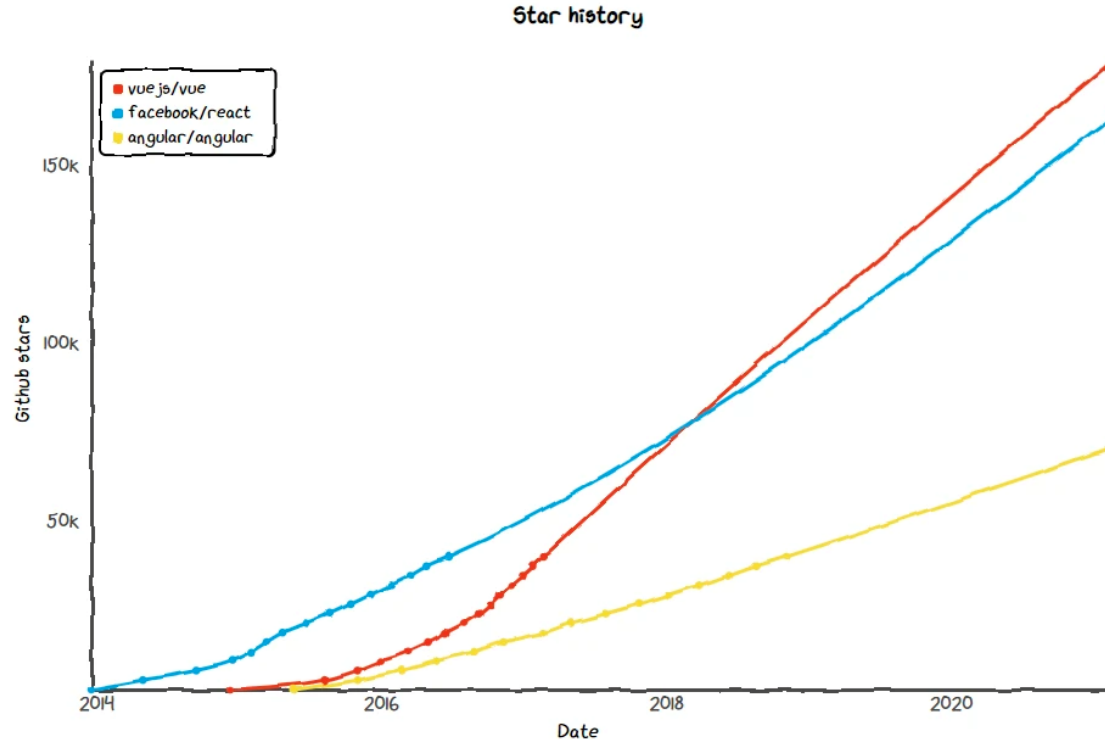


	 React	 ANGULAR	 Vue.js
	A declarative, efficient, and flexible JavaScript library for building user interfaces.	One framework. Mobile & desktop.	A progressive, incrementally adoptable JavaScript framework for building UI on the web.
	116 993 ★	59 302 ★	121 050 ★
Original author	Jordan Walke	Misko Hevery	Evan You
Developers	Facebook	Google	
Initial release	May 29, 2013	October 20, 2010	February 2014
Npm weekly downloads	3 940 035	433 361	709 943
Size	109.7 KiB production 774.7 KiB development	167 kB production 1.2 MB development	30.67 KB production 279 KB development
Easy to learn	Medium	Learn TypeScript	Yes
Coding speed	Normal	Slow	Fast
Documentation	✓	✓	✓
Performance	✓	✓	✓
Startup time	Quick	Longer due to its large codebase	Quick
Complete web apps	Needs to be integrated with many other tools	Can be used on standalone basis	Requires third party tools
Data binding	Uni-directional	Bi-directional	Bi-directional
Rendering	Server side	Client side	Server side
Model	Virtual DOM	MVC	Virtual DOM
Code reusability	No, only CSS	Yes	Yes, CSS and HTML
When to use	Production, custom UI apps	Production, esp. enterprise apps with Material UI	Startups, production
Big companies	Facebook, Yahoo, Netflix, Atlassian, KhanAcademy	Netflix, Upwork, PayPal, TheGuardian	Facebook, Alibaba, Adobe, Grammarly, GitLab

Who Use Vue?

ANGULAR.JS	REACT.JS	VUE.JS
The Guardian 	AirBnB 	Alibaba 
Upwork 	Instagram 	Grammarly 
Pay Pal 	UberEats 	IPL dashboard 
Sony 	Dropbox 	Gitlab 

Vue is Gaining Popularity in Recent Years (Based on Github Stars)



Installation of Vue.js

- Vue.js is built by design to be incrementally adoptable. This means that it can be integrated into a project multiple ways depending on the requirements.
- There are four primary ways of adding Vue.js to a project:
 - Import it as a CDN package on the page
 - Download the JavaScript files and host them yourself
 - Install it using npm
 - Use the official CLI to scaffold a project, which provides batteries-included build setups for a modern frontend workflow (e.g., hot-reload, lint-on-save, and much more)

Using CDN Method

For prototyping or learning purposes, you can use the latest version with:

```
<script src="https://unpkg.com/vue@next"></script>
```

For production, we recommend linking to a specific version number and build to avoid unexpected breakage from newer versions.

Creating an Application Instance

Every Vue application starts by creating a new application instance with the `createApp` function:

```
const app = Vue.createApp({  
  /* options */  
})
```


Hello Vue Demo

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Hello Vue</title>
5  </head>
6  <body>
7  |   <div id="hello-vue">
8  |     {{ message }}
9  |   </div>
10
11  <script src="https://unpkg.com/vue@next"></script>
12  <script>
13  |   const HelloVueApp = {
14  |     data() {
15  |       return {
16  |         message: 'Hello Vue!!'
17  |       }
18  |     }
19  |   }
20  |   Vue.createApp(HelloVueApp).mount('#hello-vue')
21  </script>
22
23  </body>
24  </html>
```

Data Properties

- The data option for a component is a function. Vue calls this function as part of creating a new component instance.
- It should return an object, which Vue will then wrap in its reactivity system and store on the component instance as \$data.

```
const app = Vue.createApp({  
  data() {  
    return { count: 4 }  
  }  
})
```

Activity: Get Started on Vue.js

- Open 1-2-activity.html, fill in the blank below.
- View the result in browser.

```
<body>
  <div id="app">
    <h1>_____</h1>

    <div class="card">
      <h2>Item</h2>
      <div>Description</div>
    </div>
  </div>
  <script src=_____></script>
  <script>
    const FlashCardApp = {
      data() {
        return {
          deckName: 'Vue.js Fundamentals'
        }
      }
    }

    Vue.createApp(FlashCardApp).mount('#app');
  </script>
</body>
```

Vue.js Fundamentals

Item

Description

Declarative Rendering

- Dynamic webpages are built when we have the facility of rendering data in the form of variables in the template.
- At the core of Vue.js is a system that enables us to declaratively render data to the DOM using straightforward template syntax:

```
<div id="counter">  
  Counter: {{ counter }}  
</div>
```

Declarative Rendering Demo

```
<body>
  <div id="counter">
    | Counter: {{ counter }}
  </div>

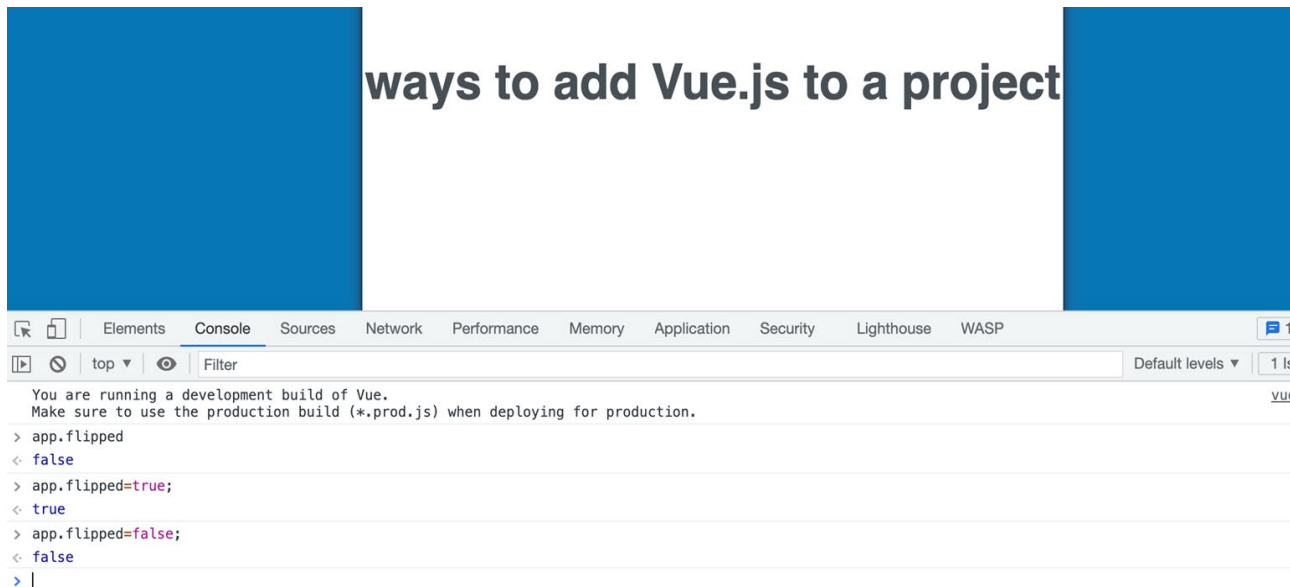
  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const Counter = {
      data() {
        | return {
          counter: 0
        }
      },
      mounted() {
        | setInterval(() => {
          |   this.counter++
          | }, 1000)
        }
      }
    }

    Vue.createApp(Counter).mount('#counter')
  </script>
```

Counter: 206

Activity: Declarative Rendering

- Open 1-4-activity.html
- Open your browser's JavaScript console) and set app.flipped to true or false. You should see the rendered example above update accordingly.



Topic 2

Using Vue Directives

- Attribute Binding
- Conditional Rendering
- List Rendering

Directives

- Directives are special attributes with the v- prefix.
- A directive's job is to reactively apply side effects to the DOM when the value of its expression change
- Example of directives
 - v-bind
 - v-if, v-else-if, v-else
 - v-show
 - v-for
 - v-on
 - v-model

Attribute Binding using v-bind

- To create a bond between an HTML element's attribute and a value from your Vue app's data, we'll use a Vue directive called v-bind. For example

```

```

- Now, we've created a reactive bond between what lives in this attribute ("image") and the image data itself.
- How exactly is the v-bind directive working? We use this directive to dynamically bind an attribute to an expression. In this case, the attribute is src and the expression is whatever is in that attribute's quotes: "image"

v-bind Arguments

- Some directives can take an "argument", denoted by a colon after the directive name.
- For example, the v-bind directive is used to reactively update an HTML attribute:
- It can take an "argument", denoted by a colon after the directive name

```
<!-- full syntax -->
<a v-bind:href="url"> ... </a>

<!-- shorthand -->
<a :href="url"> ... </a>

<!-- shorthand with dynamic argument -->
<a :[key]="url"> ... </a>
```

Examples of v-bind Arguments

```
<!-- bind an attribute -->


<!-- dynamic attribute name -->
<button v-bind:[key]="value"></button>

<!-- shorthand -->


<!-- shorthand dynamic attribute name -->
<button :[key]="value"></button>

<!-- with inline string concatenation -->


<!-- class binding -->
<div :class="{ red: isRed }"></div>
<div :class="[classA, classB]"></div>
<div :class="[classA, { classB: isB, classC: isC }]"></div>

<!-- style binding -->
<div :style="{ fontSize: size + 'px' }"></div>
<div :style="[styleObjectA, styleObjectB]"></div>
```

v-bind href Demo

```
<body>

  <div id="hello-vue">

    <h1> v-bind url</h1>
    <a href={{url}} >Search without v-bind </a> <br/>
    <a v-bind:href="url">Search with v-bind</a> <br/>
    <a v-bind:href="url + '?q=vue'">Search Vue</a><br/>
    <a :href="url + '?q=vue'">Search Vue with shorhand</a>

  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const HelloVueApp = {
      data() {
        return {
          url: "http://www.google.com/search"
        }
      }
    }

    Vue.createApp(HelloVueApp).mount('#hello-vue')
  </script>
</body>
```

v-bind url

[Search without v-bind](#)

[Search with v-bind](#)

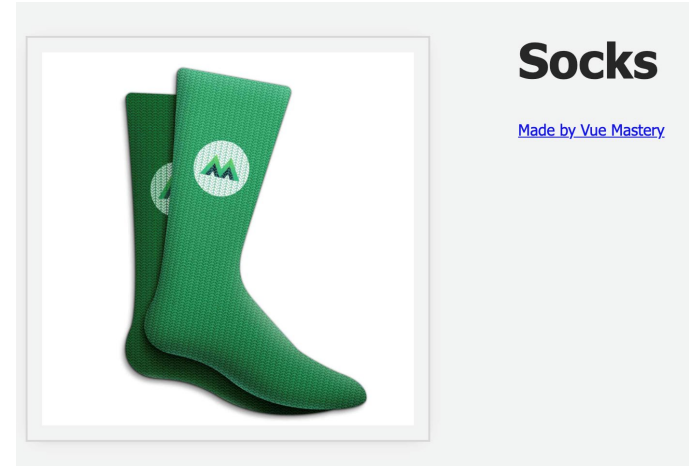
[Search Vue](#)

[Search Vue with shorhand](#)

v-bind Image Demo to show Image

```
<body>
  <div id="app">
    <div class="nav-bar"></div>
    <div class="product-display">
      <div class="product-container">
        <div class="product-image">
          
        </div>
        <div class="product-info">
          <h1>{{ product }}</h1>
          <!-- solution -->
          <a :href="url">Made by Vue Mastery</a>
          <!-- solution -->
        </div></div></div></div>

    <!-- Import App -->
    <script src="https://unpkg.com/vue@next"></script>
    <script>
      const app = Vue.createApp({
        data() {
          return {
            product: 'Socks',
            image: './assets/images/socks_green.jpg',
            // solution
            url: 'https://www.vuemastery.com/'
            // solution
          }
        }
      })
      const mountedApp = app.mount('#app')
    </script>
  </body>
```



Activity: v-bind

- Open 2-3-activity.html, fill in the blank below.
- View the result in browser.

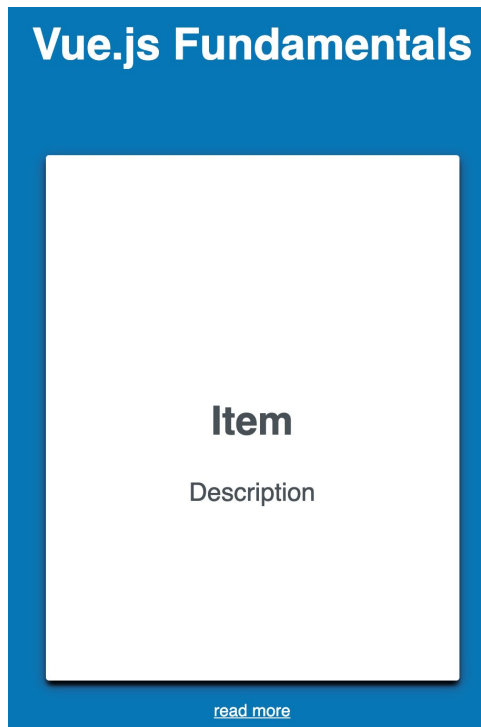
```
<body>
  <div id="app">
    <h1 v-text="deckName"></h1>

    <div class="card">
      <h2>Item</h2>
      <div>Description</div>
    </div>

    <a _____">read more</a>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const FlashCardApp = {
      data() {
        return {
          deckName: 'Vue.js Fundamentals',
          readMoreUrl: 'https://v3.vuejs.org/guide',
        };
      }
    };

    const app = Vue.createApp(FlashCardApp).mount('#app');
  </script>
</body>
```



Conditional Rendering

The directive `v-if` is used to conditionally render a block. The block will only be rendered if the directive's expression returns a truthy value.

```
<h1 v-if="awesome">Vue is awesome!</h1>  
<h1 v-else>Oh no 😞</h1>
```

v-else-if

- The v-else-if, as the name suggests, serves as an "else if block" for v-if. It can also be chained multiple times
- Similar to v-else, a v-else-if element must immediately follow a v-if or a v-else-if element.

```
<div v-if="type === 'A'">
  A
</div>
<div v-else-if="type === 'B'">
  B
</div>
<div v-else-if="type === 'C'">
  C
</div>
<div v-else>
  Not A/B/C
</div>
```


Conditional Rendering Demo

```
<div id="app">
  How many durians you eat : <input type = "text" v-model = "num">
  <br/>
  <span v-if="num > 4">You ate {{num}} durians. That is a lot</span>
  <span v-else-if="num > 0">You ate {{num}} durians</span>
  <span v-else>You have not eaten durian</span>
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
  const HelloVueApp = {
    data() {
      return {
        num: 1
      }
    }
  }
}
```

How many durians you eat :

You ate 4 durians

```
Vue.createApp(HelloVueApp).mount('#app')
```

```
</script>
```

Activity: Conditional Rendering

- Open 2-6-activity.html
- if interests = 3, print"you must be very active"
- else if interests == 0, print"please select at least one exercise"
- else show the interests

```
<div id="app">
  <input type="checkbox" v-model="_____ " value="Running"> Running
  <input type="checkbox" v-model="_____ " value="Cycling"> Cycling
  <input type="checkbox" v-model="_____ " value="Swimming"> Swimming

  <p _____="selectedInterests.length == 3">You must be very active!</p>
  <p _____="selectedInterests.length > 0">You selected: {{selectedInterests.join(', ')}}</p>
  <p _____>Please select at least one interest.</p>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        interests: ['Running', 'Cycling', 'Swimming'],
        selectedInterests: []
      }
    }
  }
  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

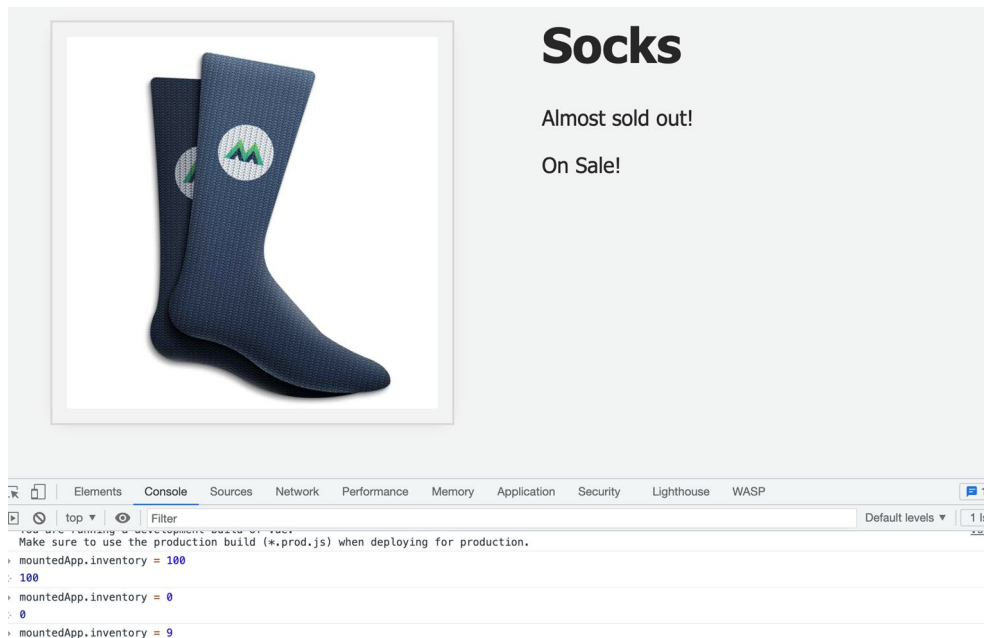
if-else-if Demo on Stock Level

```
<body>
  <div id="app">
    <div class="nav-bar"></div>
    <div class="product-display">
      <div class="product-container">
        <div class="product-image">
          
        </div>
        <div class="product-info">
          <h1>{{ product }}</h1>
          <p v-if="inventory > 10">In Stock</p>
          <p v-else-if="inventory <= 10 && inventory > 0">Almost sold out!</p>
          <p v-else>Out of Stock</p>
          <!-- solution -->
          <p v-if="onSale">On Sale!</p>
          <!-- solution -->
        </div> </div></div></div>
      <script src="https://unpkg.com/vue@next"></script>
      <script>
        const app = Vue.createApp({
          data() {
            return {
              product: 'Socks',
              image: './assets/images/socks_blue.jpg',
              inventory: 0,
              // solution
              onSale: true
              // solution
            }
          }
        })
        const mountedApp = app.mount('#app')
      </script>
    </body>
```



Activity: if-then-else on Stock Level

- Open 2-6.html
- Open your browser's JavaScript console) and change the stock value for mountedApp.inventory . You should see the rendered stock updated.



v-show

- Another option for conditionally displaying an element is the v-show directive. The usage is largely the same
- The difference is that an element with v-show will always be rendered and remain in the DOM; v-show only toggles the display CSS property of the element.

```
<h1 v-show="ok">Hello!</h1>
```

v-if vs v-show

- v-if is "real" conditional rendering because it determines whether the element exist or non-exist DOM
- In comparison, v-show is much simpler - the element is always rendered. However, the visibility of the element is toggled using CSS.

v-show Demo

```
<div id="app">
  How many durians you eat : <input type = "text" v-model = "num">
  <br/>
  <span v-show = "num > 0">You ate {{num}} durians</span>
  <span v-show = "num > 4">You ate {{num}} durians. That is a lot!</span>
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
  const HelloVueApp = {
    data() {
      return {
        num: 0
      }
    }
  }
```

How many durians you eat :

```
Vue.createApp(HelloVueApp).mount('#app')
</script>
```

Activity: v-show

Open 2-8-activity.html

Show the interests using v-show

```
<div id="app">
  <input type="checkbox" v-model="_____ " value="Running"> Running
  <input type="checkbox" v-model="_____ " value="Cycling"> Cycling
  <input type="checkbox" v-model="_____ " value="Swimming"> Swimming

  <p _____>You selected: {{selectedInterests.join(', ')}}</p>
  <p _____>Please selected one of the above</p>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        interests: ['Running','Cycling','Swimming'],
        selectedInterests: []
      }
    }
  }
  Vue.createApp(HelloVueApp).mount('#app')
</script>
```


List Rendering

- We can use the v-for directive to render a list of items based on an array.
- The v-for directive requires a special syntax in the form of item in items, where items is the source data array and item is an alias for the array element being iterated on

```
<ul id="array-rendering">
  <li v-for="item in items">
    {{ item.message }}
  </li>
</ul>
```

```
Vue.createApp({
  data() {
    return {
      items: [{ message: 'Foo' }, { message: 'Bar' }]
    }
  }
}).mount('#array-rendering')
```

v-for Demo

```
<div id="app">
  <ul>
    <li v-for="i in interests">{{i}} </li>
  </ul>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        interests: ['Running', 'Cycling', 'Swimming']
      }
    }
  }

  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

v-for Demo to List Product Details

```
<body>
<div id="app">
  <div class="nav-bar"></div>
  <div class="product-display">
    <div class="product-container">
      <div class="product-image">
        
      </div>
      <div class="product-info">
        <h1>{{ product }}</h1>
        <p v-if="inStock">In Stock</p>
        <p v-else>Out of Stock</p>
        <ul>
          <li v-for="detail in details">{{ detail }}</li>
        </ul>
        <!-- solution -->
        <ul>
          <li v-for="(size, index) in sizes" :key="index">{{ size }}</li>
        </ul>
        <!-- solution -->
        <div v-for="variant in variants" :key="variant.id">{{ variant.color }}</div>
      </div></div></div></div>
<script src="https://unpkg.com/vue@next"></script>
<script>
  const app = Vue.createApp({
    data() {
      return {
        product: 'Socks',
        image: './assets/images/socks_blue.jpg',
        inStock: true,
        details: ['50% cotton', '30% wool', '20% polyester'],
        // solution
        sizes: ['S', 'M', 'L', 'XL'],
        // solution
        variants: [
          { id: 2234, color: 'green' },
          { id: 2235, color: 'blue' },
        ]
      }
    }
  })
  const mountedApp = app.mount('#app')
</script>
```



Activity: v-for

Show the interests using v-for

```
<div id="app">
  <div _____>
    <input type="checkbox" v-model="selectedInterests" v-bind:value="_____">{{interest}}
  </div>
  <p>You selected: {{selectedInterests.join(', ')}}</p>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        interests: ['Running','Cycling','Swimming'],
        selectedInterests: []
      }
    }
  }
  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

Topic 3

Controls and Elements

Event Handling

- We can use the v-on directive, which we typically shorten to the @ symbol, to listen to DOM events and run some JavaScript when they're triggered.
- The usage would be v-on:click="methodName" or with the shortcut, @click="methodName"

```
<div id="basic-event">
  <button @click="counter += 1">Add 1</button>
  <p>The button above has been clicked {{ counter }} times.</p>
</div>
```

```
Vue.createApp({
  data() {
    return {
      counter: 0
    }
  }
}).mount('#basic-event')
```

v-on Demo

```
<div id="basic-event">  
  <button @click="counter += 1">Add 1</button>  
  <p>The button above has been clicked {{ counter }} times.</p>  
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
  Vue.createApp({  
    data() {  
      return {  
        counter: 0  
      }  
    }  
  }).mount('#basic-event')
```

```
</script>
```

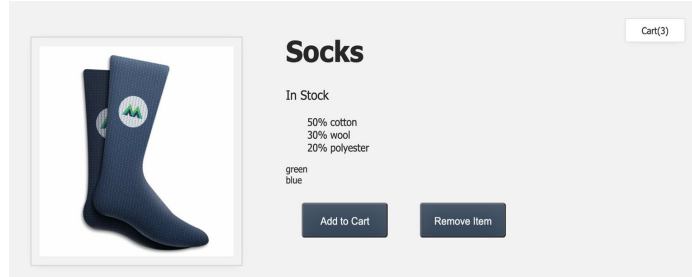
Add 1

The button above has been clicked 6 times.

Event Handling Demo on Add/Remove Cart

```
<div class="product-display">
  <div class="product-container">
    <div class="product-image">
      
    </div>
    <div class="product-info">
      <h1>{{ product }}</h1>
      <p v-if="inStock">In Stock</p>
      <p v-else>Out of Stock</p>
      <ul>
        <li v-for="detail in details">{{ detail }}</li>
      </ul>
      <div v-for="variant in variants" :key="variant.id" @mouseover="updateImage(variant.image)">{{ variant.color }}</div>
      <button class="button" v-on:click="addToCart">Add to Cart</button>
      <!-- solution -->
      <button class="button" @click="removeFromCart">Remove Item</button>
      <!-- solution -->
    </div>
  </div>
</div>
</div>
```

```
<!-- Import App -->
<script src="https://unpkg.com/vue@next"></script>
<script>
  const app = Vue.createApp({
    data() {
      return {
        cart: 0,
        product: 'Socks',
        image: './assets/images/socks_blue.jpg',
        inStock: true,
        details: ['50% cotton', '30% wool', '20% polyester'],
        variants: [
          { id: 2234, color: 'green', image: './assets/images/socks_green.jpg' },
          { id: 2235, color: 'blue', image: './assets/images/socks_blue.jpg' },
        ]
      }
    }
  })
  app.mount('#app')
```



Activity: v-on

When click, it will show the sum of 2 numbers

```
<div id="app">
  <p>First number : <input type = "text" v-model.number.lazy = "num1"></p>
  <p>Second number : <input type = "text" v-model.number.lazy = "num2"></p>
  <button _____ = "answer">Show Answer</button>
  <p v-show = "num1>0">{{num1}} + {{num2}} = {{total}}</p>
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
  const HelloVueApp = {
```

```
    data() {
```

```
      return {
```

```
        num1: 0,
```

```
        num2: 0,
```

```
        total: 0
```

```
      }
    },
```

```
    methods: {
```

```
      answer() {
```

```
        return _____
```

```
      }
    }
  }
}
```

```
Vue.createApp(HelloVueApp).mount('#app')
```

```
</script>
```

First number :

Second number :

Show Answer

10 + 20 = 30

Event Modifiers

- It is a common need to call `event.preventDefault()` or `event.stopPropagation()` inside event handlers.
- Although we can do this easily inside methods, it would be better if the methods can be purely about data logic rather than having to deal with DOM event details.
- To address this problem, Vue provides event modifiers for `v-on`. Recall that modifiers are directive postfixes denoted by a dot.
 - `.stop`
 - `.prevent`
 - `.capture`
 - `.self`
 - `.once`
 - `.passive`

Event Modifier Examples

```
<!-- the click event's propagation will be stopped -->
```

```
<a @click.stop="doThis"></a>
```

```
<!-- the submit event will no longer reload the page -->
```

```
<form @submit.prevent="onSubmit"></form>
```

```
<!-- modifiers can be chained -->
```

```
<a @click.stop.prevent="doThat"></a>
```

```
<!-- just the modifier -->
```

```
<form @submit.prevent></form>
```

Event Modifier Demo

```
<div id="app">
  <form>
    <button v-if="!loggedIn" v-on:click.prevent="logIn('Mike')">Log in</button>
    <button v-else v-on:click.prevent="logOut">Log out</button>{{msg}}
  </form>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        msg: '',
        loggedIn: false
      }
    },
    methods: {
      logIn(username) {
        this.loggedIn = true;
        this.msg = 'Welcome, ' + username + '!';
      },
      logOut() {
        this.loggedIn = false;
        this.msg = 'So long!';
      }
    }
  }

  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

Class and Style Bindings

- A common need for data binding is manipulating an element's class list and its inline styles.
- Since they are both attributes, we can use v-bind to handle them
- We can pass an object to :class (short for v-bind:class) to dynamically toggle classes:

```
<div :class="{ active: isActive }"></div>
```

```
data() {  
  return {  
    isActive: true,  
    hasError: false  
  }  
}
```

Style Binding Demo

```
<body>
  <div id="hello-vue">
    <span v-bind:style="{message}">
      {{message}}
    </span>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const HelloVueApp = {
      data() {
        return {
          message : "Hello Vue!",
          fancy: {
            backgroundColor: 'lightgrey',
            borderRadius: '10px',
            padding: '10px',
            marginTop: '30px'
          }
        }
      }
    }

    Vue.createApp(HelloVueApp).mount('#hello-vue')
  </script>
</body>
```

Hello Vue!

Class Binding Demo

```
<!DOCTYPE html>
<html>
<head>
  <title>1-1</title>
  <style>
    .salmon {
      background-color: lightsalmon;
      border-radius: 25px;
      padding: 10px;
      margin: 30px;
      font-family: 'Roboto'
    }
  </style>
</head>
<body>
  <div id="hello-vue">
    <span :class='{salmon:Issalmon}'>{{message}}</span>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const HelloVueApp = {
      data() {
        return {
          message : "Hello Vue!",
          name: 'Ally',
          Issalmon: true,
        }
      }
    }

    Vue.createApp(HelloVueApp).mount('#hello-vue')
  </script>
</body>
</html>
```

Hello Vue!

Class and Style Binding Demo on Product Color Variants

```
<div class="product-display">
  <div class="product-container">
    <div class="product-image">
      <!-- solution -->
      
      <!-- solution -->
    </div>
    <div class="product-info">
      <h1>{{ product }}</h1>
      <p v-if="inStock">In Stock</p>
      <p v-else>Out of Stock</p>
      <ul>
        <li v-for="detail in details">{{ detail }}</li>
      </ul>

      <div
        v-for="variant in variants"
        :key="variant.id"
        @mouseover="updateImage(variant.image)"
        class="color-circle"
        :style="{ backgroundColor: variant.color }">
      </div>

      <button
        class="button"
        :class="{ disabledButton: !inStock }"
        :disabled="!inStock"
        @click="addToCart">
        Add to Cart
      </button>
    </div>
  </div>
</div>
```



Activity : Class Binding

- Open 3-8-activity.html
- Add one more yellow box for warm color
- Toggle the class visibility from the browser window console.



Computed Properties

Another attribute of Vue object is computed which allows you to create more complex data Eg

```
Vue.createApp({
  data() {
    return {
      author: {
        name: 'John Doe',
        books: [
          'Vue 2 - Advanced Guide',
          'Vue 3 - Basic Guide',
          'Vue 4 - The Mystery'
        ]
      }
    }
  },
  computed: {
    // a computed getter
    publishedBooksMessage() {
      // `this` points to the vm instance
      return this.author.books.length > 0 ? 'Yes' : 'No'
    }
  }
}).mount('#computed-basics')
```

Computed Properties Demo

```
<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return{
        name: "Vue"
      }
    },
    computed: {
      welcome() {
        return "Hello " + this.name;
      }
    }
  }
  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

Hello Vue

Activity: Computed Property

- Open 3-10-activity.html
- Use Computed Property to compute Fahrenheit
- $T(^{\circ}\text{F}) = T(^{\circ}\text{C}) \times 9/5 + 32$

```
<div id="app">
  <h1>Celsius: {{ _____ }}</h1>
  <h1>Fahrenheit: {{ _____ }}</h1>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return{
        Celsius: 30
      }
    },
    computed: {
      Fahrenheit() {
        return _____
      }
    }
  }

  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

Methods

- Instead of a computed property, we can define the same function as a method.
- For the end result, the two approaches are indeed exactly the same.
- However, the difference is that computed properties are cached

```
const app = Vue.createApp({
  data() {
    return { count: 4 }
  },
  methods: {
    increment() {
      // `this` will refer to the component instance
      this.count++
    }
  }
})
```

Method Demo

```
<body>
  <div id="app">
    <h1>{{ welcome() }}</h1>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const HelloVueApp = {
      data() {
        return{
          name: "Vue"
        }
      },
      methods: {
        welcome() {
          return "Hello " + this.name;
        }
      }
    }

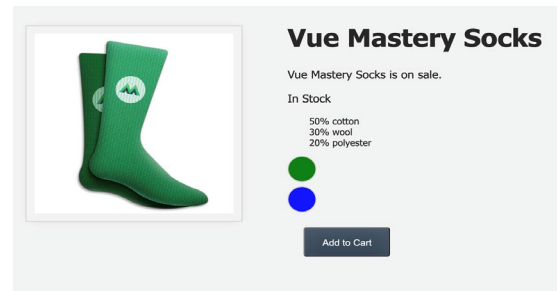
    Vue.createApp(HelloVueApp).mount('#app')
  </script>
</body>
```

Hello Vue

Computed Values and Methods

Demo on Product Info

```
methods: {
  addToCart() {
    this.cart += 1
  },
  updateVariant(index) {
    this.selectedVariant = index
  }
},
computed: {
  title() {
    return this.brand + ' ' + this.product
  },
  image() {
    return this.variants[this.selectedVariant].image
  },
  inStock() {
    return this.variants[this.selectedVariant].quantity
  },
  // solution
  saleMessage() {
    if (this.onSale) {
      return this.brand + ' ' + this.product + ' is on sale.'
    }
    return ''
  }
  // solution
}
```



Activity: Methods

- Open 3-13-activity.html
- Use Computed Property to compute Fahrenheit
- $T(^{\circ}\text{F}) = T(^{\circ}\text{C}) \times 9/5 + 32$

```
<div id="app">
  <h1>Celsius: {{ Celsius }}</h1>
  <h1>Fahrenheit: {{ _____ }}</h1>
</div>

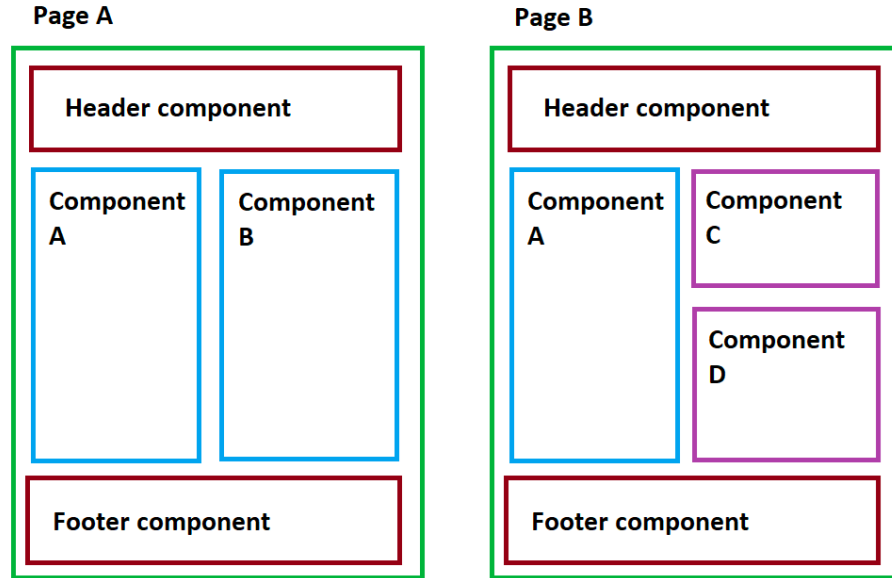
<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        Celsius: 30
      }
    },
    methods: {
      Fahrenheit() {
        return this.Celsius*9/5+32
      }
    }
  }
  Vue.createApp(HelloVueApp).mount('#app')
</script>
```


Topic 4

Components

Component

- Components are small pieces of UI, and put them together with other components to create an entire user interface.
- Components are reusable instances with a name:



Example of Component

```
// Create a Vue application
const app = Vue.createApp({})

// Define a new global component called button-counter
app.component('button-counter', {
  data() {
    return {
      count: 0
    }
  },
  template: `
    <button @click="count++">
      You clicked me {{ count }} times.
    </button>`
})
```

Component Demo

```
<div id="components-demo">
  <button-counter></button-counter>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>

  // Create a Vue application
  const app = Vue.createApp({})

  // Define a new global component called button-counter
  app.component('button-counter', {
    data() {
      return {
        count: 0
      }
    },
    template: `
      <button @click="count++">
        You clicked me {{ count }} times.
      </button>`
  })
  app.mount('#components-demo')
```

You clicked me 10 times.

Reusing Components

```
<div id="components-demo">  
  <button-counter></button-counter>  
  <button-counter></button-counter>  
  <button-counter></button-counter>  
</div>  
</div>
```

You clicked me 3 times.

You clicked me 4 times.

You clicked me 2 times.

Activity: Component

- Create a component for a blog post

```
<div id="blog-post-demo">
  _____
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>

  // Create a Vue application
  const app = Vue.createApp({})

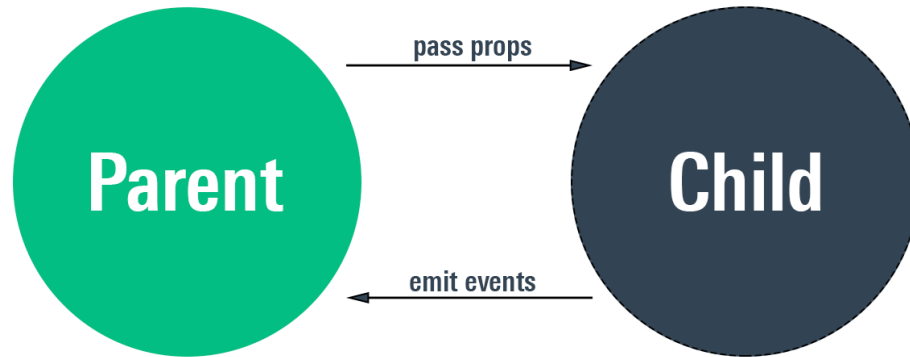
  app._____, {
    template: `<h1>My Blog</h1>`
  })

  app.mount('#blog-post-demo')

</script>
```

Passing Data to Component

- Props are custom attributes you can register on a component.
- When a value is passed to a prop attribute, it becomes a property on that component instance.
- The value of that property is accessible within the template, just like any other component property.



Prop Demo

```
<body>
  <div id="blog-post-demo" class="demo">
    <blog-post title="My journey with Vue"></blog-post>
    <blog-post title="Bloggng with Vue"></blog-post>
    <blog-post title="Why Vue is so fun"></blog-post>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>

    // Create a Vue application
    const app = Vue.createApp({})

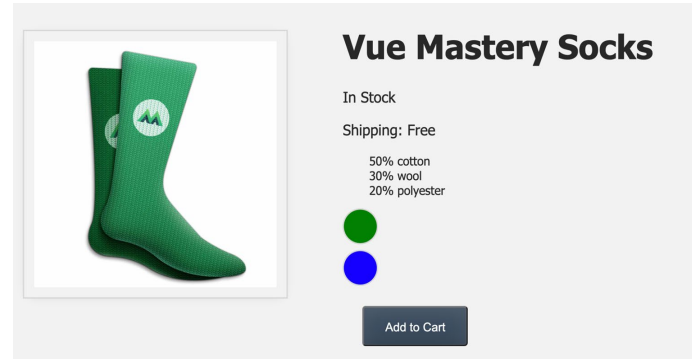
    app.component('blog-post', {
      props: ['title'],
      template: `<h4>{{ title }}</h4>`
    })
    app.mount('#blog-post-demo')
  </script>
</body>
```


Component and Props Demo on Online Product

```
<body>
  <div id="app">
    <div class="nav-bar"></div>
    <div class="cart">Cart({{ cart }})</div>
    <product-display :premium="premium"></product-display>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const app = Vue.createApp({
      data() {
        return {
          cart: 0,
          premium: true
        }
      },
      methods: {}
    })
  </script>
  <script src="./components/ProductDisplay.js"></script>
  <script src="./components/ProductDetails.js"></script>
  <script>
    const mountedApp = app.mount('#app')
  </script>
</body>
```

Add to Cart not working!
as the child component cannot
pass the data to the parent.



Activity: Props

- Open 4-6-activity.html
- Pass the todo items in the prop of the component

```
<div id="app">
  <h2>Todo list:</h2>
  <ol>
    <todo-item v-for="_____ v-bind:todo=_____ "></todo-item>
  </ol>
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
<script>

// Create a Vue application
const ToDoApp = {
  data() {
    return {
      todoList: ['Buy Groceries', 'Buy Lunch', 'Watch Movie']
    }
  }
}

const app = Vue.createApp(ToDoApp)

app.component('todo-item', {
  props: _____,
  template: ' _____ '
})

app.mount('#app')

</script>
```

Todo list:

1. Buy Groceries
2. Buy Lunch
3. Watch Movie

Emit Events

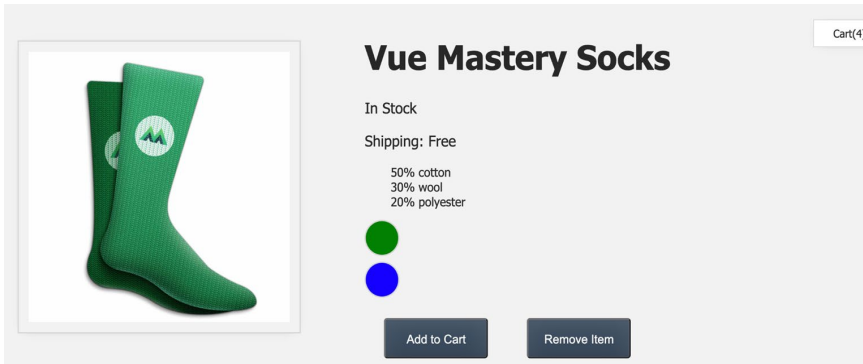
- In Vue 2, you can define the props that a component receives, but you can't declare which events it can emit.
- Vue 3 now offers an emits option, similar to the existing props option. This option can be used to define the events that a component can emit to its parent.
- Emitted events can be defined on the component via the emits option.

```
1  app.component('custom-form', {  
2    emits: ['inFocus', 'submit']  
3  })
```

Emit Event Demo on Online Product

```
},  
methods: {  
  addToCart() {  
    this.$emit('add-to-cart', this.variants[this.selectedVariant].id)  
  },  
  // solution  
  removeFromCart() {  
    this.$emit('remove-from-cart', this.variants[this.selectedVariant].id)  
  },  
  // solution  
  updateVariant(index) {  
    this.selectedVariant = index  
  }  
},  
},
```

Add to Cart is working now!
as the child component pass the data
to the parent via the emit function



Activity: Emit

- Open 4-8-activity.html
- Add the emit the option to enlarge the text upon click event.

```
<script>
const app = Vue.createApp({
  data() {
    return {
      posts: [
        { id: 1, title: 'My journey with Vue'},
        { id: 2, title: 'Blogging with Vue'},
        { id: 3, title: 'Why Vue is so fun'}
      ],
      postFontSize: 1
    }
  }
})
app.component('blog-post', {
  props: ['title'],
  template: `
    <div class="blog-post">
      <h4>{{ title }}</h4>
      <button _____('enlargeText')>
        Enlarge text
      </button>
    </div>
  `
})
app.mount('#blog-posts-events-demo')
</script>
```

My journey with Vue

Enlarge text

Blogging with Vue

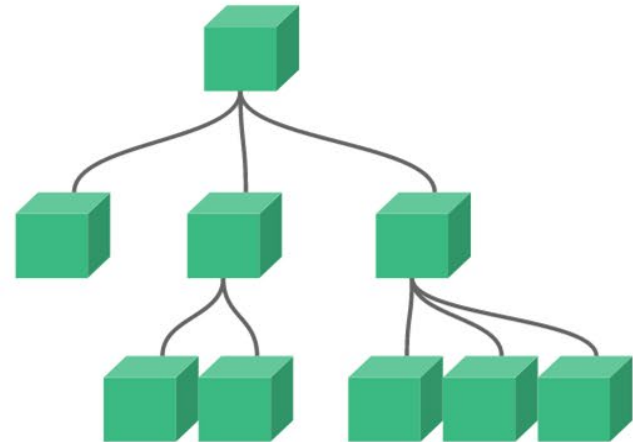
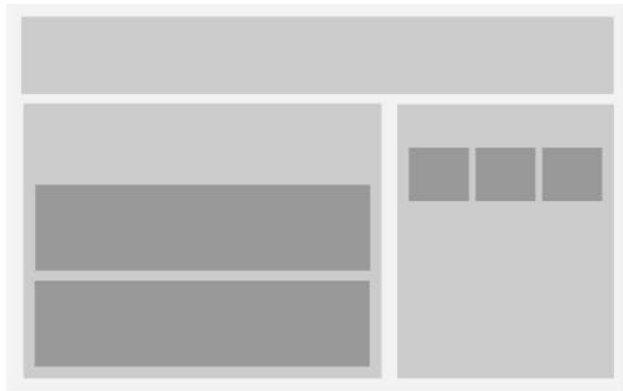
Enlarge text

Why Vue is so fun

Enlarge text

Nested (Child) Component

- It's common for an app to be organized into a tree of nested components:
- For example, you might have components for a header, sidebar, and content area, each typically containing other components for navigation links, blog posts, etc.



Nested Component Demo

```
<body>
  <div id="app">
    <welcome></welcome>
  </div>
  <script src="https://unpkg.com/vue@next"></script>
  <script>
    // Create a Vue application
    const app = Vue.createApp({})
    app.component('welcome', {
      template: '<div>\
        <hello first_name="Elon" last_name="Musk" ></hello>\
        <h1>Welcome to Vue JS Training</h1></div>'
    })
    app.component('hello', {
      props: ['first_name', 'last_name'],
      template: '<h1> Hello {{ first_name }} {{last_name}}</h1>'
    })
    app.mount('#app')
  </script>
</body>
```

Activity: Nested Component

- Open 4-10-activity.html
- Create a grocery list as shown below

```
<body>
  <div id="app">
    <grocery_____></grocery>
  </div>
  <script src="https://unpkg.com/vue@next"></script>
  <script>

    // Create a Vue application
    const GroceryApp = {
      data() {
        return {
          Tobuy_list: ['Milk', 'Bread', 'Kaya']
        }
      }
    }

    const app = Vue.createApp(GroceryApp)
    app.component('grocery', {
      props: ['grocerylist'],
      template: '
        <div>
          <h2>Grocery List:</h2>
          <ol>
            <grocery-item v-for="_____" v-bind:_____></grocery-item>
          </ol>
        </div>
      '
    })
    app.component('grocery-item', {
      props: ['buy_item'],
      template: '
        <li>{{ buy_item }}</li>
      '
    })

    app.mount('#app')
  </script>
</body>
```

Grocery List:

1. Milk
2. Bread
3. Kaya

Slot

Slots are a powerful and versatile content distribution and composition mechanism. You can think of slots as customizable templates which you can use in different places, for various use cases, producing different effects

```
<!-- todo-button component template -->  
<button class="btn-primary">  
  <slot></slot>  
</button>
```

Slot Demo

```
<div id="blog-post-demo" class="demo">
  <blog-post title="My journey with Vue">Alfred</blog-post>
  <blog-post title="Blogging with Vue">Ally</blog-post>
  <blog-post title="Why Vue is so fun">Steve</blog-post>
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
<script>
```

```
// Create a Vue application
const app = Vue.createApp({})

app.component('blog-post', {
  props: ['title'],
  template: `<h4>{{ title }}</h4>
    <i><slot></slot></i></h4>`
})
app.mount('#blog-post-demo')
</script>
```

My journey with Vue, posted by *Alfred*

Blogging with Vue, posted by *Ally*

Why Vue is so fun, posted by *Steve*

Fallback Content

- There are cases when it's useful to specify fallback (i.e. default) content for a slot, to be rendered only when no content is provided.
- For example, in a `<submit-button>` component. We might want the text "Submit" to be rendered inside the `<button>` most of the time.
- To make "Submit" the fallback content, we can place it in between the `<slot>` tags

```
<button type="submit">  
  <slot>Submit</slot>  
</button>
```

Fallback Content Demo

```
<body>
  <div id="app">
    <h2>Todo list:</h2>
    <ol>
      <todo-item v-for="item in SundayList" v-bind:todo="item">Sunday</todo-item>
      <todo-item v-for="item in MondayList" v-bind:todo="item">Monday</todo-item>
      <todo-item v-for="item in TuesdayList" v-bind:todo="item"></todo-item>
    </ol>
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>

    // Create a Vue application
    const ToDoApp = {
      data() {
        return {
          SundayList: ['Buy Groceries', 'Watch Movie', 'Visit Mom'],
          MondayList: ['Presentation', 'Pickup Laundry'],
          TuesdayList: ['Medical Appointment', 'Send Laundry']
        }
      }
    }

    const app = Vue.createApp(ToDoApp)

    app.component('todo-item', {
      props: ['todo'],
      template: '<li>{{todo}} on <slot>Tuesday</slot></li>'
    })

    app.mount('#app')

  </script>
</body>
```

Todo list:

1. Buy Groceries on Sunday
2. Watch Movie on Sunday
3. Visit Mom on Sunday
4. Presentation on Monday
5. Pickup Laundry on Monday
6. Medical Appointment on Tuesday
7. Send Laundry on Tuesday

Activity: Slot

- Open 4-13-activity.html
- Create a todo list as shown below

```
<body>
  <div id="app">
    <h2>Todo list:</h2>
    <ol>
      <todo-item v-for="item in SundayList" v-bind:todo="item">_____</todo-item>
      <todo-item v-for="item in MondayList" v-bind:todo="item">_____</todo-item>
    </ol>
  </div>
  <script src="https://unpkg.com/vue@next"></script>
  <script>
    // Create a Vue application
    const ToDoApp = {
      data() {
        return {
          SundayList: ['Buy Groceries','Watch Movie','Visit Mom'],
          MondayList: ['Presentation','Pickup Laundry']
        }
      }
    }

    const app = Vue.createApp(ToDoApp)
    app.component('todo-item', {
      props: ['todo'],
      template: '<li>{{todo}} on _____</li>'
    })
    app.mount('#app')
  </script>
</body>
```

Todo list:

1. Buy Groceries on Sunday
2. Watch Movie on Sunday
3. Visit Mom on Sunday
4. Presentation on Monday
5. Pickup Laundry on Monday

Topic 5

Create User Defined Forms

Form Input Bindings

- You can use the v-model directive to create two-way data bindings on form input, textarea, and select elements.
- It automatically picks the correct way to update the element based on the input type

```
<input v-model="message" placeholder="edit me" />
<p>Message is: {{ message }}</p>
```

```
<!-- bad -->
<textarea>{{ text }}</textarea>

<!-- good -->
<textarea v-model="text"></textarea>
```

v-model Demo

```
<body>
  <div id="app">
    Name: <input type='text' v-model="name">
    <p>Hello {{ name }}</p>

    Hobby: <input type='text' v-model.lazy="hobby">
    <p>Hello {{ hobby }}</p>

  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const HelloVueApp = {
      data() {
        return {
          name: '',
          hobby: '',
          url: "http://www.google.com/search"
        }
      }
    }

    Vue.createApp(HelloVueApp).mount('#app')
  </script>
```

Name:

Hello

Hobby:

Hello

Modifiers

- Modifiers modify the behavior of the v-model.
- There are 3 modifiers for v-model:
 - lazy - syncs the input with the data after change
 - number - automatically typecast as a number
 - trim - user input to be trimmed automatically

```
<!-- synced after "change" instead of "input" -->  
<input v-model.lazy="msg" />
```

```
<input v-model.number="age" type="text" />
```

```
<input v-model.trim="msg" />
```

Activity: v-model

- Open 5-2-activity.html
- Capture the Celsius data from the input box.
- Apply lazy and number modifiers

```
<div id="app">
  <p> Celsius: <input type='text' _____.number= _____ > </p>
  <p> Fahrenheit: _____ </p>
</div>

<script src="https://unpkg.com/vue@next"></script>
<script>
  const HelloVueApp = {
    data() {
      return {
        celsius: 30,
      }
    },
    computed: {
      fahrenheit () {
        return this.celsius*9/5 + 32;
      }
    }
  }

  Vue.createApp(HelloVueApp).mount('#app')
</script>
```

Watch

- While computed properties are more appropriate in most cases, there are times when a custom watcher is necessary.
- Watch property in Vue provides a more generic way to observe and react to data changes on a Vue instance:

Watch Demo

Kilometers :

Meters :

```
<div id="app">
  Kilometers : <input type = "text" v-model = "kilometers">
  Meters : <input type = "text" v-model = "meters">
</div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
  const HelloVueApp = {
    data() {
      return {
        kilometers : 0,
        meters:0
      }
    },
    watch : {
      kilometers(val) {
        this.kilometers = val;
        this.meters = val * 1000;
      },
      meters(val) {
        this.kilometers = val/ 1000;
        this.meters = val;
      }
    }
  }
```

```
Vue.createApp(HelloVueApp).mount('#app')
```

```
</script>
```

Activity: Watch

- Open 5-3-activity.html
- Capture the Celsius and Fahrenheit data from the input box using watch.

```
<body>
  <div id="app">
    Celsius : <input type = "text" v-model = "_____ ">
    Fahrenheit : <input type = "text" v-model = "_____ ">
  </div>

  <script src="https://unpkg.com/vue@next"></script>
  <script>
    const HelloVueApp = {
      data() {
        return {
          celsius : 0,
          fahrenheit:32.0
        }
      },
      methods: {
        celsius(val) {
          this.celsius = _____
          this.fahrenheit = Math.round(val*9/5+32,1);
        },
        fahrenheit(val) {
          this.celsius = _____
          this.fahrenheit = val;
        }
      }
    }

    Vue.createApp(HelloVueApp).mount('#app')
  </script>
</body>
```

Checkbox v-model Demo

```
<body>
  <div id="v-model-multiple-checkboxes">
    <input type="checkbox" id="jack" value="Jack" v-model="checkedNames" />
    <label for="jack">Jack</label>
    <input type="checkbox" id="john" value="John" v-model="checkedNames" />
    <label for="john">John</label>
    <input type="checkbox" id="mike" value="Mike" v-model="checkedNames" />
    <label for="mike">Mike</label>
    <br />
    <span>Checked names: {{ checkedNames }}</span>
  </div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
  Vue.createApp({
    data() {
      return {
        checkedNames: []
      }
    }
  }).mount('#v-model-multiple-checkboxes')
```

☒ Jack ☒ John ☒ Mike
Checked names: ["Jack", "John", "Mike"]

```
</script>
```

```
</body>
```

Radio v-model Demo

```
<body>
  <div id="v-model-radiobutton">
    <input type="radio" id="one" value="One" v-model="picked" />
    <label for="one">One</label>
    <br />
    <input type="radio" id="two" value="Two" v-model="picked" />
    <label for="two">Two</label>
    <br />
    <span>Picked: {{ picked }}</span>
  </div>
```

```
<script src="https://unpkg.com/vue@next"></script>
```

```
<script>
```

```
Vue.createApp({
  data() {
    return {
      picked: ''
    }
  }
})
```

```
).mount('#v-model-radiobutton')
```

```
</script>
```

```
</body>
```

☐ One

☒ Two

Picked: Two

Select v-model Demo

```
<body>
  <div id="v-model-select" class="demo">
    <select v-model="selected">
      <option disabled value="">Please select one</option>
      <option>A</option>
      <option>B</option>
      <option>C</option>
    </select>
    <span>Selected: {{ selected }}</span>
  </div>
```

```
<script src="https://unpkg.com/vue@next"></script>
<script>
  Vue.createApp({
    data() {
      return {
        selected: ''
      }
    }
  }).mount('#v-model-select')
</script>
</body>
```

B ▼ Selected: B

Dynamic options rendered with v-for

```
<body>
  <div id="v-model-select-dynamic" class="demo">
    <select v-model="selected">
      <option v-for="option in options" :value="option.value">
        | {{ option.text }}
      </option>
    </select>
    <span>Selected: {{ selected }}</span>
  </div>
  <script src="https://unpkg.com/vue@next"></script>
  <script>
    Vue.createApp({
      data() {
        return {
          selected: 'A',
          options: [
            { text: 'One', value: 'A' },
            { text: 'Two', value: 'B' },
            { text: 'Three', value: 'C' }
          ]
        }
      }
    }).mount('#v-model-select-dynamic')
  </script>
</body>
```

Three ▼ Selected: C

Form Binding Demo For Product Review

```
</template>`  
<form class="review-form" @submit.prevent="onSubmit">  
  <h3>Leave a review</h3>  
  <label for="name">Name:</label>  
  <input id="name" v-model="name">  
  
  <label for="review">Review:</label>  
  <textarea id="review" v-model="review"></textarea>  
  
  <label for="rating">Rating:</label>  
  <select id="rating" v-model.number="rating">  
    <option>5</option>  
    <option>4</option>  
    <option>3</option>  
    <option>2</option>  
    <option>1</option>  
  </select>  
  
  <input class="button" type="submit" value="Submit">  
  
</form>`,
```

Reviews:

Alfred gave this 5 stars
"test"

Leave a review

Name:

Review:

Rating:

Submit

Activity: Form Binding

- Open 5-10-activity.html
- Modify the ReviewForm2.js and ReviewList2.js to add a recommend (Yes/No) field.

Reviews:

Alfred gave this 5 stars
"Test"
Recommended: Yes

Leave a review

Name:

Review:

Rating:

Would you recommend this product?

Vue Cli

- Vue provides an official CLI for quickly scaffolding ambitious Single Page Applications (SPA).
- Vue Cli is a full system for rapid Vue.js development platform. It allows you to create Vue App can also be deployed on the server
- Vue Cli is based on node.js

This course will not cover Vue Cli and SPA

Installing Vue CLI

- Download and Install Node JS
<https://nodejs.org/en/download/>
- To install Vue CLI, type the following command to your Windows CMD Prompt or Mac Terminal

```
npm install -g @vue/cli
```

Create Vue Project

To create Vue project, type the following command to your Windows CMD Prompt or Mac Terminal:

```
vue create <<project-name>.
```

```
cd <<project-name>>
```

```
npm run serve
```

Vue App

View the vue app on the browser <http://localhost:8080/>



Welcome to Your Vue.js App

For a guide and recipes on how to configure / customize this project,
check out the [vue-cli documentation](#).

Installed CLI Plugins

[babel](#) [eslint](#)

Essential Links

[Core Docs](#) [Forum](#) [Community Chat](#) [Twitter](#) [News](#)

Ecosystem

[vue-router](#) [vuex](#) [vue-devtools](#) [vue-loader](#) [awesome-vue](#)

Summary Q&A

Cert & TRAQOM Survey (Mandatory)

<https://lms-tms.tertiaryinfotech.com/>

Certificate Delivery



Scan to receive your certificate

<https://goo.gl/R2eumq>

Open Link

Copy

Course Ref: **TGS-2025052468**

TRAQOM Survey



Your feedback helps us improve

https://ssgtraqom.qualtrics.com/jfe/form/SV_3K9i7rTJ9OLsauW?Q_CHL=qr

Open Link

Copy

Course Run ID: **1322326**

Digital Attendance

- It is mandatory for you to take both AM, PM and Assessment digital attendance for WSQ funded courses.
- The trainer or administrator will show you the digital attendance QR code generated from SSG portal.
- Please scan the QR code from your mobile phone camera and submit your attendance.

Final Assessment

Recommended Courses

1. [WSQ - Mastering HTML5 and CSS3 Fundamentals for Professional Web Design](#)
2. [WSQ - Hands-On Web App Development with Javascript](#)
3. [WSQ - React for Beginners - Build Your First Web App](#)
4. [WSQ - Design and Build Responsive Websites from Scratch](#)
5. [WSQ – Mastering NoSQL Fundamentals for Beginners](#)

Support

If you have any enquiries during and after the class, you can contact us below

Email: enquiry@tertiaryinfotech.com

Tel: 61000613

Thank You!